

Go-Around Classification Using ADS-B and METAR Data

BBL514E – Pattern Recognition Term Project

Furkan Güney 704241023
Alper Berkin Yazıcı 704241020

Istanbul Technical University
Department of Computer Science

May 2026

Abstract

This study addressed the binary classification of aircraft go-arounds, which are aborted landing attempts followed by a climb from final approach. Go-arounds are rare but operationally important events because they increase pilot and air traffic controller workload, reduce runway efficiency, and may contribute to delays in dense terminal airspace. The study used publicly available Automatic Dependent Surveillance–Broadcast trajectory records augmented with METAR surface weather observations. The dataset contained approximately 9 million landing attempts from 176 airports during 2019, with approximately 33,000 go-around events, corresponding to a positive-class prevalence of about 0.37%. Five classifier families were evaluated: Linear Discriminant Analysis, Logistic Regression, Random Forest, LightGBM, and a Multi-Layer Perceptron. Two feature sets were compared: an operational context-only feature set and an extended feature set including METAR weather variables. A temporal train-validation-test split was used to prevent leakage across time. The best model according to validation precision-recall AUC was the Multi-Layer Perceptron trained on the context and METAR feature set. On the held-out test set, it achieved a ROC-AUC of 0.6846 and a PR-AUC of 0.0160. Weather features improved performance across all classifier families. The final model was deployed in a Dockerized system with a FastAPI backend and an HTML web interface for real-time single-flight prediction.

Keywords: pattern recognition, go-around prediction, ADS-B, METAR, imbalanced classification, aviation safety, machine learning

1 Introduction and Literature Review

1.1 Problem Motivation

A go-around is a safety-critical maneuver in which a flight crew discontinues a landing attempt and initiates a missed approach or climb-out procedure. Although go-arounds are normal and safe procedures when stabilized landing criteria cannot be satisfied, they have important operational consequences. They increase pilot and air traffic controller workload, disrupt landing sequences, consume additional fuel, and may reduce runway throughput in busy terminal areas. Therefore, even a probabilistic early-warning system that identifies elevated go-around risk may be useful for arrival management and air traffic planning.

From a data-driven perspective, go-around prediction is challenging because the event is extremely rare. In the dataset used in this project, go-arounds constitute only about 0.37% of

all landing attempts. This means that a trivial classifier predicting every flight as a normal landing would achieve more than 99% accuracy while being useless for detecting the positive class. For this reason, the task requires evaluation metrics that are sensitive to minority-class performance, especially precision, recall, F1-score, ROC-AUC, and precision-recall AUC.

1.2 Pattern Recognition Perspective

The task is formulated as a binary pattern recognition problem on tabular flight-level features. Each landing attempt is represented by a feature vector containing operational context, temporal information, airport and runway identifiers, aircraft type, and meteorological observations. The goal is to learn a mapping from this feature vector to the probability that the landing attempt will result in a go-around.

The problem includes several classical pattern recognition difficulties. First, the class distribution is severely imbalanced. Second, the feature space is heterogeneous, containing both numerical variables such as wind speed and visibility and categorical variables such as airport, aircraft type, and runway. Third, the classes are not easily separable because many adverse weather or operational conditions also occur during normal landings. Finally, the model must produce meaningful probability scores rather than only hard labels, because the operational value of the system lies in risk ranking.

1.3 Related Work

Prior research has shown that go-arounds can be studied using open aviation data sources. Proud [1] demonstrated that go-arounds can be detected automatically from crowd-sourced ADS-B position data by analyzing altitude profiles during final approach. Figuet et al. [2] extended this direction to prediction and showed that machine learning models trained on ADS-B and open weather data can identify flights with elevated go-around risk. Monstein et al. [3] published the large-scale landing trajectory dataset used in this project, enabling reproducible research on go-around analysis.

Kumar et al. [4] formulated go-around occurrence as a supervised classification problem using ADS-B data and reported that aircraft type, runway characteristics, and temporal patterns carry predictive information. Dhief et al. [5] proposed a machine-learned go-around prediction model using pilot-in-the-loop simulation data and emphasized the importance of calibration and imbalance-aware evaluation. Liu et al. [6] developed a real-time go-around prediction system for JFK Airport, showing the practical relevance of deployment-oriented prediction tools.

The classifier families used in this project are also well established in the pattern recognition literature. Fisher’s Linear Discriminant Analysis [7] provides a generative linear baseline under Gaussian class-conditional assumptions with equal covariance matrices. Logistic Regression is a discriminative probabilistic classifier and is widely used as a calibrated linear baseline [8]. Random Forests [9] reduce variance through bootstrap aggregation of decision trees and are effective for heterogeneous tabular data. LightGBM [10] is an efficient gradient-boosted tree framework using histogram-based split finding and leaf-wise growth. Multi-Layer Perceptrons are feedforward neural networks capable of modeling nonlinear relationships when trained with sufficient data [11].

Class imbalance is a central issue in this project. He and Garcia [12] surveyed imbalanced learning methods and showed that class weighting and cost-sensitive learning are common approaches for rare-event classification. Davis and Goadrich [13] argued that precision-recall curves are more informative than ROC curves when the positive class is rare, because PR curves directly expose

precision degradation under severe imbalance.

1.4 Objective and Contributions

The objective of this project is to evaluate whether go-around events can be predicted from publicly available ADS-B landing records and METAR weather information at the per-flight level. The main contributions are as follows:

1. A reproducible end-to-end machine learning pipeline for go-around classification on a large-scale public aviation dataset.
2. A controlled ablation study comparing operational context-only features against context plus METAR weather features.
3. A temporal evaluation protocol that separates training, validation, and test periods to reduce leakage across time.
4. A Dockerized model-serving system with a FastAPI backend and an HTML interface for real-time single-flight prediction.

2 Materials and Methods

2.1 Dataset Description

The primary data source is the *Large Landing Trajectory Dataset for Go-Around Analysis* [3]. The dataset provides one row per landing attempt and includes pre-computed go-around labels, airport and runway information, aircraft identifiers, trajectory-derived attributes, and METAR weather variables.

Table 1: Dataset summary.

Property	Value
Total landings	Approximately 9,000,000
Go-around events	Approximately 33,000
Positive-class prevalence	Approximately 0.37%
Airports	176
Countries	44
Year of data	2019
Classes	Normal landing, go-around
Feature sets	Context only / context + METAR

The class distribution is highly imbalanced. Since go-arounds are much rarer than normal landings, accuracy alone is not a suitable performance measure. A classifier that always predicts the negative class would achieve very high accuracy but zero recall for go-arounds.

2.2 Feature Engineering

Two feature sets were constructed in order to quantify the contribution of weather information.

Feature Set 1: Context Only. This feature set includes operational, temporal, and categorical context variables. Numerical variables include glide slope angle, runway length, month, day of week, and hour in UTC. Categorical variables include airport, runway, aircraft type code, ICAO aircraft type, wake turbulence category, intersection indicator, airport country, airport region, operator country, and operator region.

Feature Set 2: Context + METAR. This feature set extends the context-only variables with meteorological observations. Numerical METAR variables include wind speed, wind direction, wind gust, visibility, temperature, sea-level pressure, and station pressure. Categorical weather variables include weather intensity, precipitation, weather description, obscuration, and other weather indicators.

2.3 Preprocessing

The preprocessing pipeline was designed to handle both numerical and categorical variables. Missing numerical values were imputed using the median value computed on the training data, and numerical features were standardized to zero mean and unit variance. Missing categorical values were imputed using the most frequent category or an explicit UNKNOWN category. Categorical features were one-hot encoded.

To control dimensionality, rare categorical levels appearing fewer than 2,000 times in the training data were grouped into an infrequent category. This is particularly important because airport, runway, aircraft type, and operator-related variables may contain many levels.

The columns `n_approaches` and `n_rwy_approached` were excluded from the model input. These variables encode information about how many approaches a flight eventually made, which is post-hoc information. Including them would leak the target because a go-around generally implies an additional approach.

2.4 Mathematical Formulation

Let the dataset be

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \quad (1)$$

where $x_i \in \mathbb{R}^d$ is the feature vector of the i -th landing attempt and $y_i \in \{0, 1\}$ is the binary label. The label $y_i = 1$ denotes a go-around and $y_i = 0$ denotes a normal landing.

The goal is to learn a scoring function

$$f : \mathbb{R}^d \rightarrow [0, 1], \quad (2)$$

where $f(x_i)$ estimates the posterior probability

$$f(x_i) \approx P(y_i = 1 \mid x_i). \quad (3)$$

A hard decision is produced using a threshold τ :

$$\hat{y}_i = \begin{cases} 1, & f(x_i) \geq \tau, \\ 0, & f(x_i) < \tau. \end{cases} \quad (4)$$

The threshold was tuned on the validation set to maximize F1-score:

$$\tau^* = \arg \max_{\tau} \frac{2P(\tau)R(\tau)}{P(\tau) + R(\tau)}, \quad (5)$$

where $P(\tau)$ and $R(\tau)$ are precision and recall at threshold τ .

2.5 Model Descriptions

2.5.1 Linear Discriminant Analysis

Linear Discriminant Analysis assumes that class-conditional feature distributions are Gaussian with a shared covariance matrix:

$$p(x \mid y = k) = \mathcal{N}(x \mid \mu_k, \Sigma), \quad k \in \{0, 1\}. \quad (6)$$

Under this assumption, the log-posterior odds are linear in x , and the classifier can be expressed as

$$P(y = 1 \mid x) = \sigma(w^T x + b), \quad (7)$$

where $\sigma(\cdot)$ is the sigmoid function. LDA was used as a generative linear baseline.

2.5.2 Logistic Regression

Logistic Regression directly models the posterior probability as

$$P(y_i = 1 \mid x_i) = \sigma(w^T x_i + b), \quad (8)$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (9)$$

The parameters were estimated by minimizing class-weighted binary cross-entropy:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N w_{y_i} [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]. \quad (10)$$

Class weights were used to reduce the dominance of the majority class.

2.5.3 Random Forest

Random Forest is an ensemble of decision trees trained using bootstrap samples and random feature subsets at each split. If $\hat{p}_t(x)$ is the probability predicted by tree t , the ensemble prediction is

$$\hat{p}(x) = \frac{1}{T} \sum_{t=1}^T \hat{p}_t(x). \quad (11)$$

In this project, balanced class weights were used to compensate for the rare positive class.

2.5.4 LightGBM

LightGBM is a gradient-boosted decision tree framework optimized for large-scale tabular datasets. It uses histogram-based split finding and leaf-wise tree growth. The model minimizes an additive objective consisting of a loss term and a regularization term:

$$\mathcal{J} = \sum_{i=1}^N \ell(y_i, \hat{y}_i) + \sum_{m=1}^M \Omega(g_m), \quad (12)$$

where g_m is the m -th tree. The imbalance parameter `scale_pos_weight` was set according to the ratio of negative to positive samples.

2.5.5 Multi-Layer Perceptron

The Multi-Layer Perceptron is a feedforward neural network that maps the input feature vector to a probability score. The implemented network used two hidden layers with ReLU activations:

$$h^{(1)} = \text{ReLU}(W^{(1)}x + b^{(1)}), \quad (13)$$

$$h^{(2)} = \text{ReLU}(W^{(2)}h^{(1)} + b^{(2)}), \quad (14)$$

$$\hat{p} = \sigma(W^{(3)}h^{(2)} + b^{(3)}). \quad (15)$$

The model was trained using binary cross-entropy and the Adam optimizer. Early stopping was applied to reduce overfitting.

3 Experimental Setup

3.1 Temporal Split

A temporal split was used to prevent leakage across time. This is important because random splitting could allow temporally correlated airport, weather, or traffic patterns to appear in both training and test data.

Table 2: Temporal train-validation-test split.

Split	Period	Rows
Training	January–August 2019	Approximately 5,664,824
Validation	September–October 2019	Approximately 1,648,722
Test	November–December 2019	Approximately 1,652,663

Model selection and threshold tuning were performed only on the validation set. The test set was used only for final evaluation.

3.2 Training Protocol

All positive training samples were retained. To manage memory constraints, a stratified 20% subsample of negative training samples was used. This produced a training set of approximately 1,148,594 rows with 19,537 go-around examples. Validation and test sets were evaluated at full size.

3.3 Hyperparameter and Threshold Selection

For models supporting class weighting, weights were set inversely proportional to class frequencies. LightGBM used `scale_pos_weight`. The MLP used early stopping. The final classification threshold was selected from the validation precision-recall curve by maximizing F1-score. The selected threshold for the final model was approximately $\tau = 0.13$.

3.4 Evaluation Metrics

The following metrics were computed:

- **Accuracy:**

$$\frac{TP + TN}{TP + TN + FP + FN} \quad (16)$$

- **Precision:**

$$\frac{TP}{TP + FP} \quad (17)$$

- **Recall:**

$$\frac{TP}{TP + FN} \quad (18)$$

- **F1-score:**

$$\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (19)$$

- **ROC-AUC:** area under the receiver operating characteristic curve.
- **PR-AUC:** area under the precision-recall curve, used as the primary ranking metric because the positive class is rare.

3.5 Software Environment

Table 3: Software environment.

Component	Version / Tool
Programming language	Python 3.12
Machine learning	scikit-learn 1.4+, LightGBM 4.x
Data processing	pandas, polars
Backend	FastAPI, Uvicorn
Deployment	Docker
Container base image	python:3.11-slim

4 System and Deployment

In addition to the academic classification pipeline, the project was implemented as a deployable prediction system. The trained model was exposed through a FastAPI backend. The backend receives flight-level feature inputs, applies the same preprocessing pipeline used during training, and returns the predicted class and estimated go-around probability.

A simple HTML-based web interface was developed to interact with the backend. The interface accepts user input, sends a request to the prediction endpoint, and displays the prediction result together with the model confidence or probability. The backend and frontend are served from the same Docker container, satisfying the deployment requirement of the project.

The system can be built and executed with Docker commands similar to the following:

```
docker build -t goaround-classification .
docker run -p 8000:8000 goaround-classification
```

After the container starts, the web interface can be accessed through the browser, and the API can be tested through the prediction endpoint. This deployment structure enables a live demo in which the user enters flight and weather features and receives a real-time go-around risk prediction.

5 Results

5.1 Model Comparison

Table 4 presents validation and test metrics for all evaluated model configurations. Models are ordered according to validation PR-AUC, which was the primary model selection criterion.

Table 4: Model comparison using tuned thresholds.

Model	Feature Set	Val ROC-AUC	Val PR-AUC	Test ROC-AUC	Test PR-AUC	Test F1
MLP	context + METAR	0.6833	0.0179	0.6846	0.0160	0.043
LightGBM	context + METAR	0.6840	0.0158	0.6788	0.0134	0.030
MLP	context only	0.6157	0.0157	0.6092	0.0123	0.037
LightGBM	context only	0.6250	0.0127	0.5731	0.0097	0.031
Random Forest	context + METAR	0.6807	0.0119	0.6896	0.0115	0.037
Random Forest	context only	0.6329	0.0114	0.6394	0.0106	0.032
LDA	context + METAR	0.6710	0.0102	0.6814	0.0100	0.034
Logistic Regression	context + METAR	0.6779	0.0098	0.6908	0.0102	0.033
LDA	context only	0.6184	0.0077	0.6256	0.0075	0.024
Logistic Regression	context only	0.6242	0.0072	0.6293	0.0070	0.025

The MLP trained on the context + METAR feature set achieved the highest validation PR-AUC and was selected as the final model. Although Logistic Regression and Random Forest achieved slightly higher test ROC-AUC values, their PR-AUC values were lower. Since the positive class is extremely rare, PR-AUC is more informative than ROC-AUC for evaluating the ability to retrieve go-around cases.

All PR-AUC values are low in absolute terms, which reflects the difficulty of the task. However, the best model’s PR-AUC of 0.0160 is substantially above the no-skill baseline implied by the positive prevalence of approximately 0.0035.

5.2 Confusion Matrix

The confusion matrix of the final model on the held-out test set is shown in Figure 1. The threshold was tuned on the validation set and set to approximately 0.13.

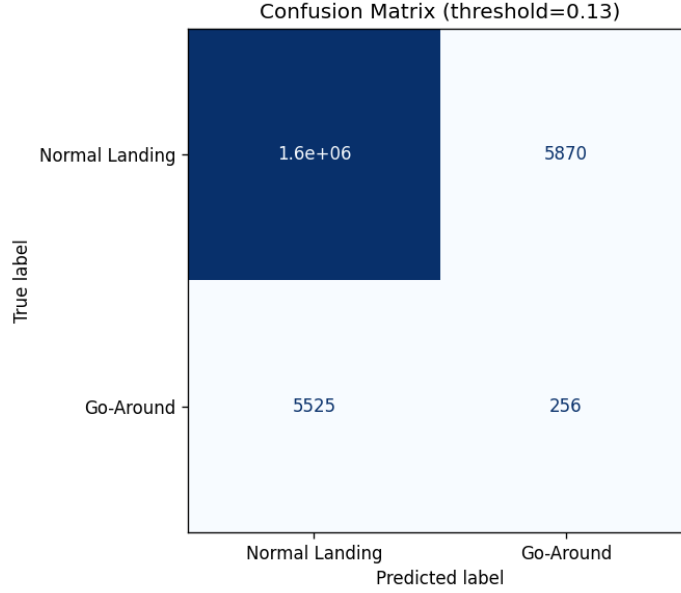


Figure 1: Confusion matrix of the final MLP model on the test set at threshold $\tau \approx 0.13$.

Table 5: Confusion matrix values for the final model.

	Predicted Normal	Predicted Go-Around
Actual Normal	1,641,012	5,870
Actual Go-Around	5,525	256

From this confusion matrix, the final model achieved a precision of 4.18%, a recall of 4.43%, and an accuracy of 99.31%. The high accuracy is misleading because the majority class dominates the dataset. The more meaningful interpretation is that the model can rank some go-around cases above normal landings, but reliable detection remains difficult due to the severe class imbalance and overlapping feature distributions.

5.3 ROC and Precision-Recall Curves

Figure 2 shows the ROC and precision-recall curves of the final model.

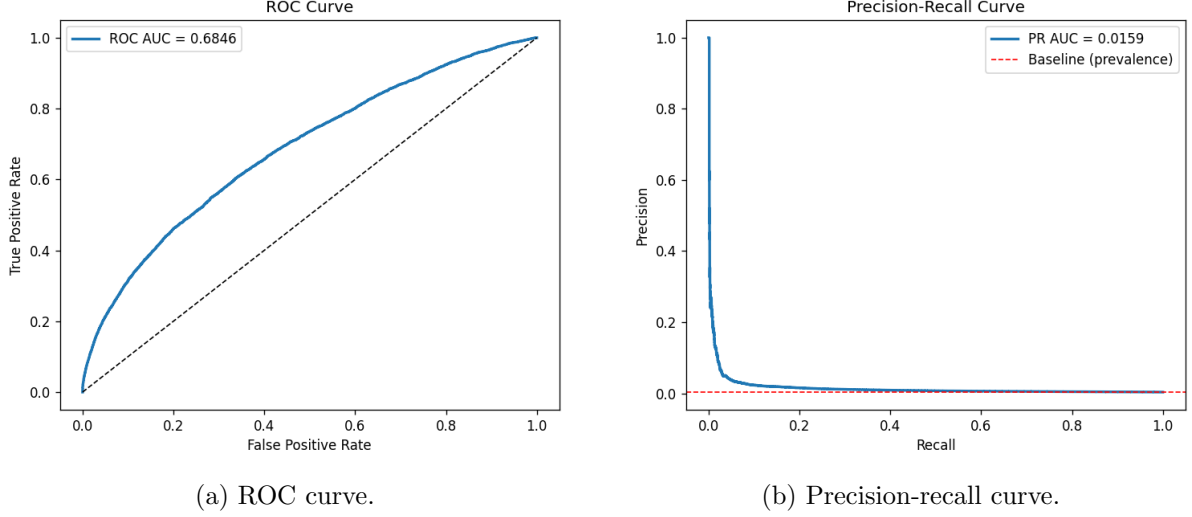


Figure 2: ROC and precision-recall curves of the final model.

The ROC-AUC of 0.6846 indicates moderate ranking ability. However, the PR-AUC of 0.0160 shows that precision remains low even when the model improves over the no-skill baseline. This gap between ROC-AUC and PR-AUC is typical in rare-event classification problems: ROC curves can look moderately good even when the number of false positives is operationally significant.

5.4 Threshold and Calibration Analysis

Figure 3 presents threshold-dependent precision, recall, and F1 behavior together with the calibration curve.

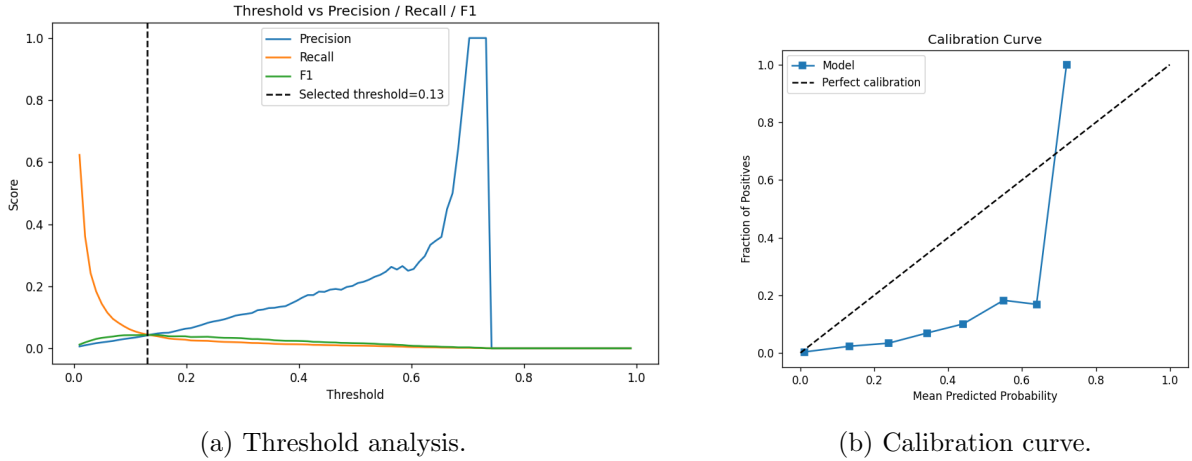


Figure 3: Threshold selection and probability calibration analysis.

The threshold analysis shows that the F1-maximizing threshold is much lower than the default value of 0.5. This is expected because the model is trained on a highly imbalanced dataset, and a high threshold would classify almost all flights as normal landings. The calibration curve indicates that the predicted probabilities should be interpreted as relative risk scores rather than perfectly calibrated probabilities. In particular, the model can rank risk to some extent, but its raw probability estimates may require post-hoc calibration before operational deployment.

5.5 Ablation Study: Contribution of METAR Features

The ablation study compares the context-only and context + METAR feature sets. Table 6 shows that adding weather information improved PR-AUC for every classifier family.

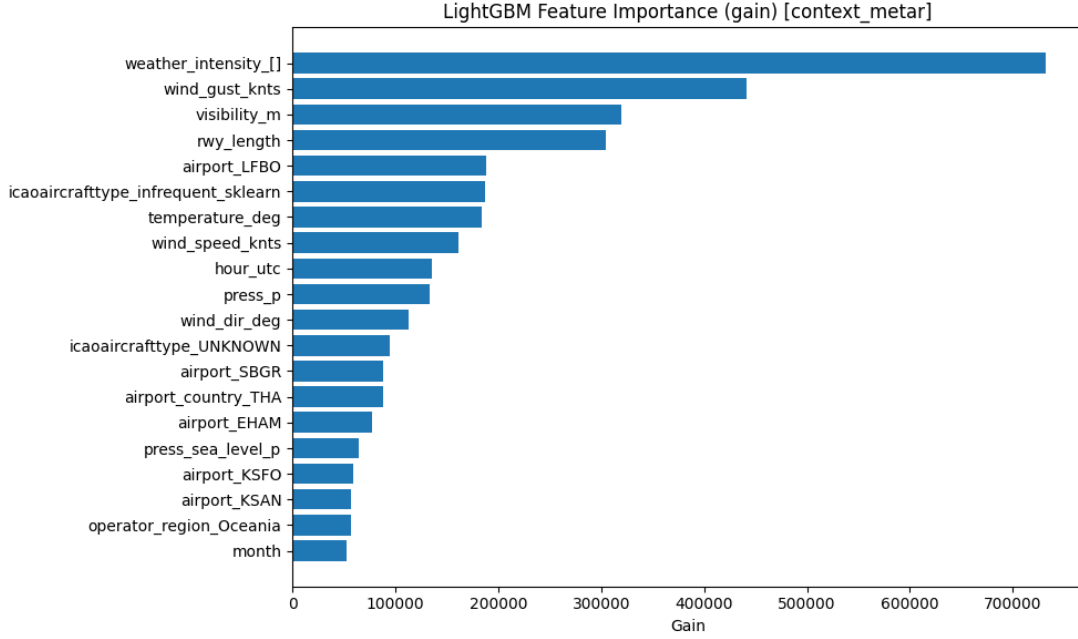
Table 6: Ablation study showing the contribution of METAR weather features.

Model	PR-AUC Context Only	PR-AUC Context + METAR	Relative Gain
MLP	0.0123	0.0160	+30%
LightGBM	0.0097	0.0134	+38%
Random Forest	0.0106	0.0115	+8%
LDA	0.0075	0.0100	+33%
Logistic Regression	0.0070	0.0102	+46%

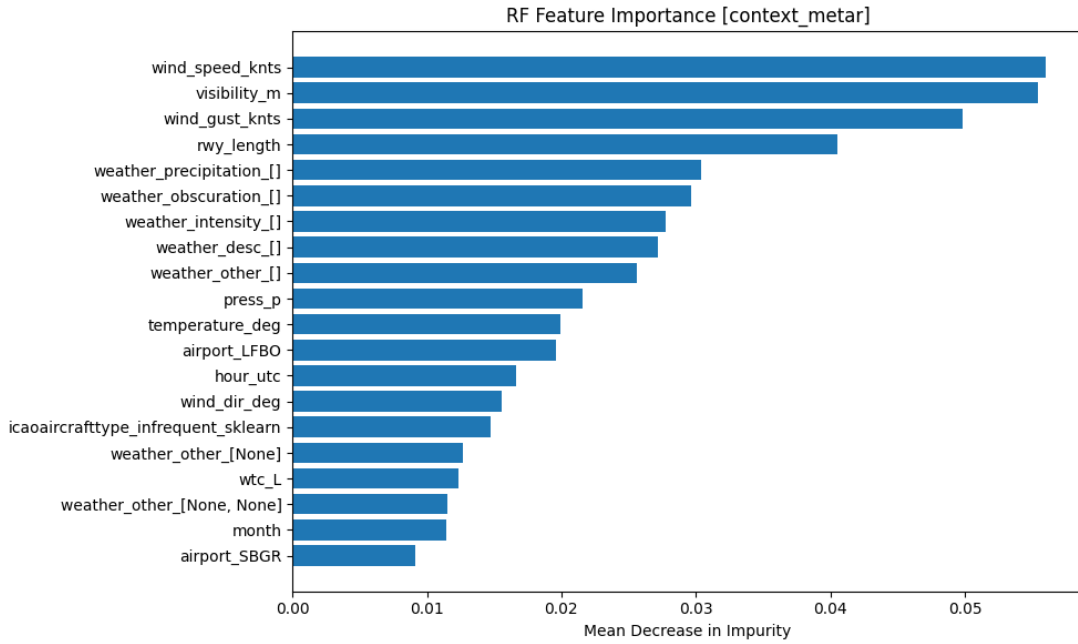
The consistent improvement across all model families confirms that meteorological variables provide additional predictive signal beyond airport, runway, aircraft, and time-related context. Wind, visibility, pressure, and weather-code variables are therefore relevant for estimating go-around risk.

5.6 Feature Importance

Feature importance plots for LightGBM and Random Forest are shown in Figure 4. Although the final selected model was the MLP, tree-based feature importance provides useful interpretability for the tabular feature space.



(a) LightGBM feature importance using gain.



(b) Random Forest feature importance using mean decrease in impurity.

Figure 4: Feature importance for context + METAR models.

The LightGBM model assigns high importance to weather intensity, wind gust, visibility, runway length, airport, aircraft type, temperature, and wind speed. Random Forest also emphasizes wind speed, visibility, wind gust, runway length, and weather-related variables. These findings are consistent with the ablation study: meteorological conditions are among the most informative predictors of go-around risk.

5.7 Error Analysis

The final model produced 5,525 false negatives and 5,870 false positives on the test set. False negatives correspond to missed go-around events, while false positives correspond to normal landings flagged as go-around risk.

Missed go-arounds were concentrated at high-traffic airports such as KORD, KPHL, KSFO, EGLL, and KDFW. This is partly expected because airports with more arrivals naturally have more go-around events in absolute terms. False positives were frequent at airports such as KLGA, SBBR, YSSY, LFBO, and LTFJ. These airports may have runway configurations, traffic patterns, or local weather conditions that resemble go-around-associated contexts without actually resulting in a go-around.

6 Discussion

The results show that go-around classification from open ADS-B and METAR data is feasible but difficult. The best model achieved a test ROC-AUC of 0.6846, indicating that it learned a non-random ranking of flights by go-around risk. However, the test PR-AUC of 0.0160 and the low precision and recall values show that direct binary detection remains limited.

A major reason for this limitation is the rarity of the event. With a positive rate below 0.4%, even a small false positive rate can produce thousands of false alarms. Another reason is feature overlap: many flights experience wind, reduced visibility, or operational complexity but still land normally. Therefore, the available flight-level aggregate features are not sufficient to clearly separate go-arounds from normal landings.

The ablation study provides one of the strongest empirical findings of the project. Adding METAR features improved PR-AUC for every classifier family. This suggests that weather conditions contain independent information that is not captured by airport, aircraft, runway, and temporal context alone. Nevertheless, the absolute performance remains limited, indicating that richer trajectory features may be necessary.

The comparison between classifiers suggests that model choice matters, but feature representation is the primary bottleneck. Linear models, tree-based models, and the MLP achieved broadly similar ROC-AUC values on the full feature set. This implies that the available variables contain only moderate discriminative signal, and future improvements are more likely to come from better features, temporal sequence modeling, and airport-specific calibration than from changing the classifier alone.

7 Conclusion

This project studied go-around classification as a binary pattern recognition problem using large-scale ADS-B landing records and METAR weather observations. Five classifier families were evaluated across two feature sets under a temporal train-validation-test protocol. The best-performing model according to validation PR-AUC was the Multi-Layer Perceptron trained on the combined context + METAR feature set. On the held-out test set, it achieved a ROC-AUC of 0.6846 and a PR-AUC of 0.0160, approximately several times higher than the no-skill baseline implied by the positive-class prevalence.

The most important empirical finding is that METAR weather features consistently improved performance across all classifiers. Wind, visibility, weather intensity, temperature, and pressure variables provided useful information beyond operational context. However, the low absolute

PR-AUC, precision, and recall values show that go-around prediction remains a difficult rare-event classification problem.

The project also satisfied the system requirements by deploying the trained model inside a Docker container with a FastAPI backend and an HTML web interface. This enabled real-time single-flight prediction and made the system suitable for live demonstration.

The main limitation is that the feature representation uses flight-level aggregate variables rather than full time-series trajectory information. Future work should incorporate per-second ADS-B sequences, including altitude, vertical rate, ground speed, and distance-to-runway profiles. Sequence models such as LSTMs or Transformers may capture approach instability patterns that are lost in aggregate features. Additional improvements may include probability calibration, focal loss, cost-sensitive boosting, airport-specific models, and full-data training without negative subsampling.

References

- [1] S. R. Proud, “Go-Around Detection Using Crowd-Sourced ADS-B Position Data,” *Aerospace*, vol. 7, no. 2, p. 16, 2020, doi: 10.3390/aerospace7020016.
- [2] B. Figuet, R. Monstein, M. Waltert, and S. Barry, “Predicting Airplane Go-Arounds Using Machine Learning and Open-Source Data,” *Proceedings*, vol. 59, no. 1, p. 6, 2020, doi: 10.3390/proceedings2020059006.
- [3] R. Monstein, B. Figuet, T. Krauth, M. Waltert, and M. Dettling, “Large Landing Trajectory Dataset for Go-Around Analysis,” *Engineering Proceedings*, vol. 28, no. 1, p. 2, 2022, doi: 10.3390/engproc2022028002.
- [4] S. G. Kumar, S. J. Corrado, T. G. Puranik, and D. N. Mavris, “Classification and Analysis of Go-Arounds in Commercial Aviation Using ADS-B Data,” *Aerospace*, vol. 8, no. 10, p. 291, 2021, doi: 10.3390/aerospace8100291.
- [5] I. Dhief, S. Alam, N. Lilith, and C. C. Mean, “A Machine Learned Go-Around Prediction Model Using Pilot-in-the-Loop Simulations,” *Transportation Research Part C: Emerging Technologies*, vol. 140, art. 103704, 2022, doi: 10.1016/j.trc.2022.103704.
- [6] K. Liu, K. Ding, L. Dai, M. Hansen, K. Chan, and J. Schade, “Real-Time Go-Around Prediction: A Case Study of JFK Airport,” *arXiv preprint arXiv:2405.12244*, 2024, doi: 10.48550/arXiv.2405.12244.
- [7] R. A. Fisher, “The Use of Multiple Measurements in Taxonomic Problems,” *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936, doi: 10.1111/j.1469-1809.1936.tb02137.x.
- [8] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [9] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.
- [10] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.
- [11] Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015, doi: 10.1038/nature14539.

- [12] H. He and E. A. Garcia, “Learning from Imbalanced Data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009, doi: 10.1109/TKDE.2008.239.
- [13] J. Davis and M. Goadrich, “The Relationship Between Precision-Recall and ROC Curves,” in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 233–240, doi: 10.1145/1143844.1143874.